

The University of Azad Jammu and Kashmir, Muzaffarabad

Name: Muneeba Arshad

Roll NO: 2024-SE-06

Course Title: Digital Image Processing

Instructor Name: Engr. Fahad Nisar

Lab: 02

Semester: 5th

Session: 2024-2028 (SE)

Department of Software Engineering

✓ Lab Report

1. Introduction

Digital Image Processing is a technique used to manipulate, analyze, and improve digital images using computer-based methods. A digital image consists of small elements called pixels, and each pixel contains information about color and intensity.

In this laboratory task, basic digital image processing operations were performed using Python in Anaconda Jupyter Notebook. The practical work included loading an image, examining pixel values, identifying color channels, converting images into grayscale, cropping and resizing images, generating histograms, and understanding different bit depths.

2. Objectives

The main objectives of this laboratory task were:

- To set up the Python environment using Anaconda and Jupyter Notebook.
- To understand the structure and properties of digital images.
- To load and inspect images using OpenCV.
- To access individual pixel values and color channels.
- To convert images from BGR to RGB and grayscale.
- To perform image cropping and resizing.
- To generate and interpret grayscale histograms.
- To understand image resolution and bit depth.
- To perform image quantization using different bit depths.

3. Tools and Technologies

The following tools and libraries were used:

Tool / Library	Purpose
Anaconda Navigator	Python environment management
Jupyter Notebook	Writing and executing Python code
Python 3	Programming language

OpenCV	Image loading and processing
NumPy	Numerical operations and pixel manipulation
Matplotlib	Image visualization and plotting

4. Environment Setup

Anaconda Navigator was used to launch Jupyter Notebook. A project folder named dip-lab-02 was created to organize the laboratory files.

A new Python notebook named lab02.ipynb was created. The required Python libraries were installed using the following command:

Installation of numpy, opencv-python, and matplotlib

```
%pip install numpy opencv-python matplotlib
```

```
Requirement already satisfied: numpy in D:\Anaconda3\Lib\site-packages (2.4.6)
Requirement already satisfied: opencv-python in D:\Anaconda3\Lib\site-packages (5.0.0.93)
Requirement already satisfied: matplotlib in D:\Anaconda3\Lib\site-packages (3.11.0)
Requirement already satisfied: contourpy>=1.0.1 in D:\Anaconda3\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in D:\Anaconda3\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in D:\Anaconda3\Lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in D:\Anaconda3\Lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in D:\Anaconda3\Lib\site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=9 in D:\Anaconda3\Lib\site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in D:\Anaconda3\Lib\site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in D:\Anaconda3\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in D:\Anaconda3\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

5. Importing Required Libraries

The required libraries were imported into the notebook.

Imports Laibrary

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

print("OpenCV version:", cv.__version__)
print("NumPy version:", np.__version__)
print("Libraries imported successfully!")
```

```
OpenCV version: 5.0.0
NumPy version: 2.4.6
Libraries imported successfully!
```

OpenCV was imported using the alias cv, NumPy using np, and Matplotlib using plt. These libraries provided the functions required for image processing, numerical calculations, and visualization.

6. Loading the Input Image

A personal image named my_image.png was selected and loaded using OpenCV's imread() function.

Loading The Dgital Image

```
[20]: image_path = "my_image.png"

img = cv.imread(image_path)

if img is None:
    raise FileNotFoundError(
        f"Image not found: {image_path}"
    )

print("Image loaded successfully!")
print("Image shape:", img.shape)
print("Image data type:", img.dtype)
```

```
Image loaded successfully!
Image shape: (1024, 1536, 3)
Image data type: uint8
```

The imread() function reads the image from the specified file path. The shape attribute provides the image dimensions and number of channels, while dtype identifies the data type of pixel values.

7. Extracting Image Dimensions

The height, width, and number of color channels were extracted using the image's shape.

Inspecting Image (Print the height, width, and number of channels)

```
[3]: height, width, channels = img.shape

print("Image Height:", height)
print("Image Width:", width)
print("Number of Channels:", channels)
```

```
Image Height: 1024
Image Width: 1536
Number of Channels: 3
```

The image dimensions describe its spatial resolution. A color image generally contains three channels: Blue, Green, and Red.

8. Accessing Pixel Values

Individual pixel values were accessed using NumPy array indexing.

Inspecting Image (Print the top-left and center pixels using [row, column] indexing.)

```
[8]: print("Top-left pixel (BGR):", img[0, 0])

     center_y = height // 2
     center_x = width // 2

     print("Center coordinates (row, column):",
           (center_y, center_x))

     print("Center pixel (BGR):",
           img[center_y, center_x])
```

```
Top-left pixel (BGR): [131 117 105]
Center coordinates (row, column): (512, 768)
Center pixel (BGR): [64 50 38]
```

The top-left pixel was accessed using coordinates [0, 0]. The center pixel was obtained by calculating half of the image height and width.

9. Minimum and Maximum Pixel Values

The minimum and maximum pixel intensities were calculated for the entire image and individual color channels.

Inspecting Image (Print the global minimum/maximum and per-channel minimum/maximum values)

```
]: print("Global minimum:", img.min())
   print("Global maximum:", img.max())

   print("Per-channel minimum (B, G, R):",
         img.min(axis=(0, 1)))

   print("Per-channel maximum (B, G, R):",
         img.max(axis=(0, 1)))
```

```
Global minimum: 0
Global maximum: 255
Per-channel minimum (B, G, R): [0 0 0]
Per-channel maximum (B, G, R): [255 255 255]
```

For an 8-bit image, pixel intensity values generally range from 0 to 255. A value of 0 represents minimum intensity, while 255 represents maximum intensity.

10. Color Space Conversion

OpenCV loads color images in BGR format, whereas Matplotlib normally displays color images in RGB format. Therefore, the image was converted from BGR to RGB. A grayscale version was also generated.

Color Space Conversion

```
6]: rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)

    gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

    red = rgb[:, :, 0]
    green = rgb[:, :, 1]
    blue = rgb[:, :, 2]

    print("RGB shape:", rgb.shape)
    print("Grayscale shape:", gray.shape)
```

```
RGB shape: (1024, 1536, 3)
Grayscale shape: (1024, 1536)
```

The RGB format represents colors using Red, Green, and Blue channels. The grayscale image contains intensity information rather than three separate color channels.

11. Extracting Individual Color Channels

Extracting Individual Color Channels

```
[7]: red = rgb[:, :, 0]
     green = rgb[:, :, 1]
     blue = rgb[:, :, 2]

     print("Red Channel Shape:", red.shape)
     print("Green Channel Shape:", green.shape)
     print("Blue Channel Shape:", blue.shape)
```

```
Red Channel Shape: (1024, 1536)
Green Channel Shape: (1024, 1536)
Blue Channel Shape: (1024, 1536)
```

Each channel represents the intensity of one primary color component in the image.

12. Visualizing Color Channels

The original RGB image, individual color channels, grayscale image, and original BGR array were displayed using Matplotlib.

Visualizing Original Image and Color Channels

```
]: fig, axes = plt.subplots(2, 3, figsize=(14, 8))

   axes[0, 0].imshow(rgb)
   axes[0, 0].set_title("Original RGB")

   axes[0, 1].imshow(red, cmap="gray")
   axes[0, 1].set_title("Red Channel")

   axes[0, 2].imshow(green, cmap="gray")
   axes[0, 2].set_title("Green Channel")

   axes[1, 0].imshow(blue, cmap="gray")
   axes[1, 0].set_title("Blue Channel")

   axes[1, 1].imshow(gray, cmap="gray")
   axes[1, 1].set_title("Grayscale")

   axes[1, 2].imshow(img)
   axes[1, 2].set_title("Unconverted BGR Display")
```

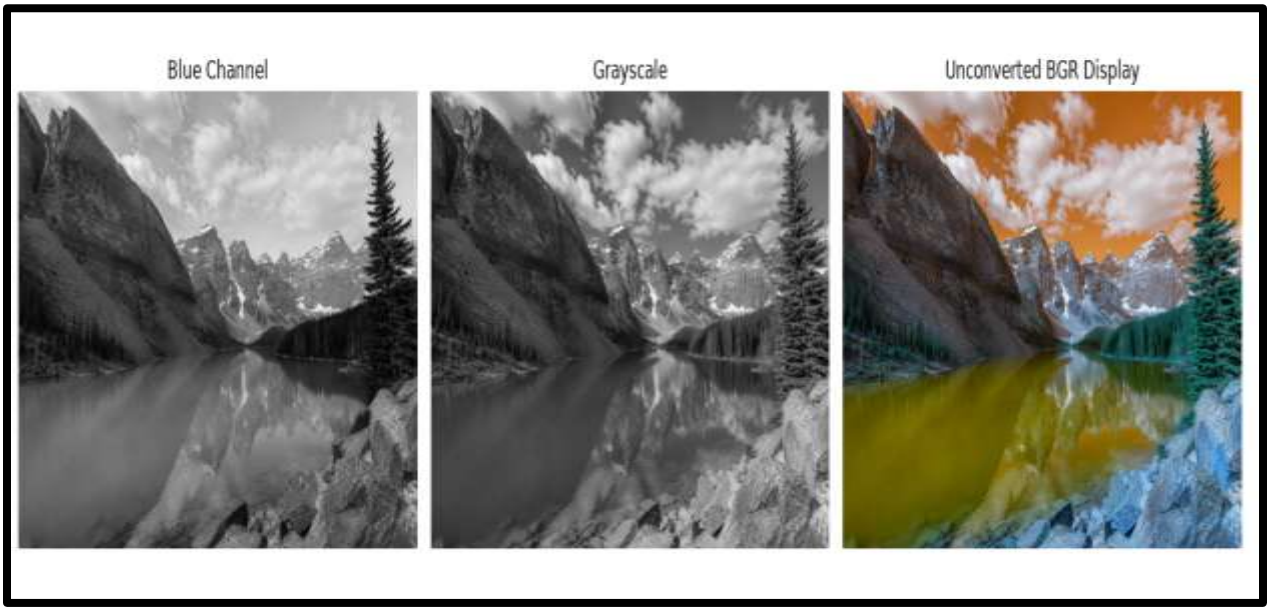
```
axes[1, 2].imshow(img)
axes[1, 2].set_title("Unconverted BGR Display")

for ax in axes.flat:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

The six images were displayed in a 2×3 layout. The individual channels were displayed in grayscale to show their intensity distribution.





13. Extracting RGB and HEX Values

The RGB value of the center pixel was extracted and converted into its hexadecimal color representation.

Extracting Center Pixel RGB and HEX Values ¶

```
height, width, channels = img.shape

center_y = height // 2
center_x = width // 2

print("Center Y:", center_y)
print("Center X:", center_x)

center_rgb = rgb[center_y, center_x]

r, g, b = map(int, center_rgb)

hex_color = f"#{r:02X}{g:02X}{b:02X}"

print("Center pixel RGB:", (r, g, b))
print("Center pixel HEX:", hex_color)
```

```
Center Y: 512
Center X: 768
Center pixel RGB: (38, 50, 64)
Center pixel HEX: #263240
```

he RGB representation contains three color intensity values, while the HEX representation expresses the same color using hexadecimal notation.

14. Image Cropping

A 100×100 pixel region was cropped from the center of the original image.

Image Cropping and Resizing

[13]:

```
y_start = (height - 100) // 2
x_start = (width - 100) // 2

crop_rgb = rgb[
    y_start:y_start + 100,
    x_start:x_start + 100
]

print("Crop shape:", crop_rgb.shape)
```

```
Crop shape: (100, 100, 3)
```

Image cropping extracts a selected region from the original image. The resulting cropped image has dimensions of 100×100 pixels.

15. Image Resizing

The cropped image was resized from 100×100 pixels to 200×200 pixels using OpenCV.

Resizing the Cropped Image (200×200)

```
[14]: resized_rgb = cv.resize(
          crop_rgb,
          (200, 200),
          interpolation=cv.INTER_LINEAR
      )

      print("Resized shape:", resized_rgb.shape)
```

```
Resized shape: (200, 200, 3)
```

Resizing changes the dimensions of an image. In this task, the cropped image was enlarged to twice its original width and height.

16. Comparing Original, Cropped, and Resized Images

The original, cropped, and resized images were displayed side by side.

Comparing Original, Cropped, and Resized Images

```
[15]: fig, axes = plt.subplots(1, 3, figsize=(13, 5))

      axes[0].imshow(rgb)
      axes[0].set_title("Original Image")

      axes[1].imshow(crop_rgb)
      axes[1].set_title("100 x 100 Crop")

      axes[2].imshow(resized_rgb)
      axes[2].set_title("200 x 200 Resized")

      for ax in axes:
          ax.axis("off")

      plt.tight_layout()
      plt.show()
```

The comparison demonstrates how cropping extracts a smaller region and resizing changes the dimensions of that region.



17. Saving the Resized Image

The resized image was converted back to BGR format and saved as resized_crop.png.

Saving the Resized Image

```
[16]: resized_bgr = cv.cvtColor(
          resized_rgb,
          cv.COLOR_RGB2BGR
      )

      saved = cv.imwrite(
          "resized_crop.png",
          resized_bgr
      )

      print("Image saved successfully:", saved)
```

```
Image saved successfully: True
```

The imwrite() function saves the processed image to the project folder. A True result indicates that the image was successfully saved.

18. Grayscale Histogram

A grayscale histogram was generated to analyze the distribution of pixel intensity values.

Grayscale Histogram

```
[17]: gray_flat = gray.flatten()

plt.figure(figsize=(10, 5))

plt.hist(
    gray_flat,
    bins=256,
    range=(0, 256),
    color="gray"
)

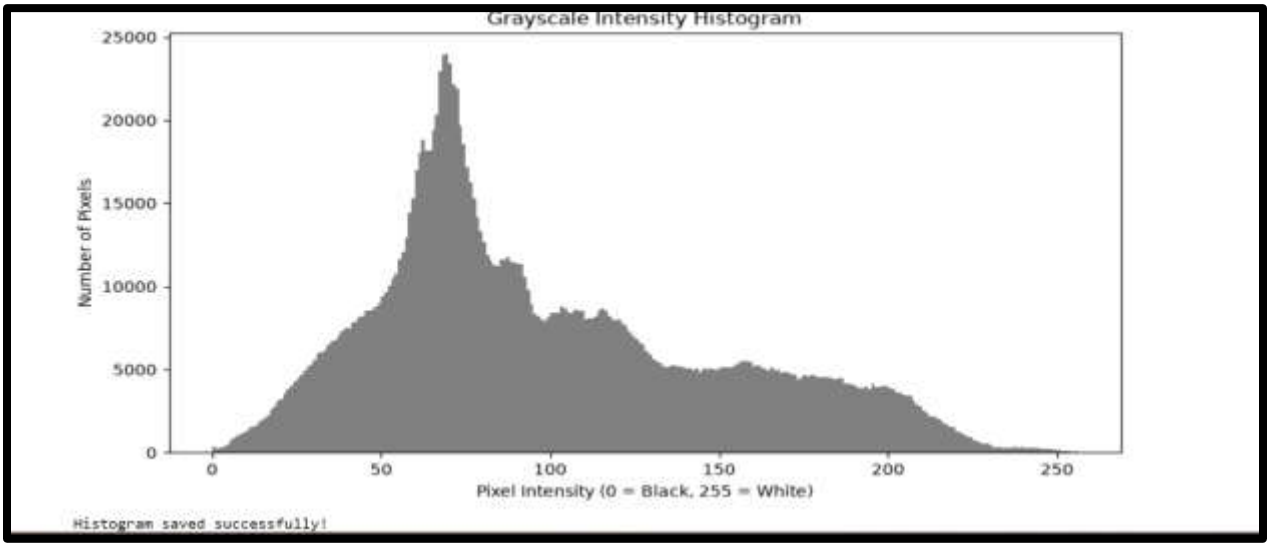
plt.title("Grayscale Intensity Histogram")
plt.xlabel("Pixel Intensity (0 = Black, 255 = White)")
plt.ylabel("Number of Pixels")

plt.savefig(
    "grayscale_histogram.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

print("Histogram saved successfully!")
```

The histogram represents the frequency of grayscale intensity values in the image. The horizontal axis represents pixel intensity from 0 to 255, while the vertical axis represents the number of pixels at each intensity level.



Histogram Interpretation

The histogram shows that most pixel intensities are concentrated between 50 and 100, with the highest peak around 65–75. This indicates that the image contains mostly dark and mid-tone regions, while fewer pixels have high intensity values. The distribution extends towards brighter intensities, but very bright pixels (near 255) are limited.

19. Generating an 8-Bit Grayscale Gradient

An 8-bit grayscale gradient was generated using NumPy.

Bit Depth and Quantization

Generating an 8-Bit Grayscale Gradient

```
[18]: gradient = np.arange(256, dtype=np.uint8)

      gradient_image = np.tile(gradient, (50, 1))

      print("Gradient shape:", gradient_image.shape)
      print("Unique values:", len(np.unique(gradient)))
```

```
Gradient shape: (50, 256)
Unique values: 256
```

20. Bit Depth Quantization

The gradient was quantized into 1-bit, 2-bit, and 4-bit representations.

Quantizing Gradients to 1-Bit, 2-Bit, and 4-Bit

[19]:
```
def quantize_gradient(gradient, bits):
    levels = 2 ** bits

    quantized = np.round(
        gradient.astype(np.float32)
        * (levels - 1) / 255
    )

    quantized = np.round(
        quantized * 255 / (levels - 1)
    )

    return quantized.astype(np.uint8)


gradient_1bit = quantize_gradient(gradient, 1)
gradient_2bit = quantize_gradient(gradient, 2)
gradient_4bit = quantize_gradient(gradient, 4)

print("8-bit:", len(np.unique(gradient)))
```

```
    return quantized.astype(np.uint8)


gradient_1bit = quantize_gradient(gradient, 1)
gradient_2bit = quantize_gradient(gradient, 2)
gradient_4bit = quantize_gradient(gradient, 4)

print("8-bit:", len(np.unique(gradient)))
print("1-bit:", len(np.unique(gradient_1bit)))
print("2-bit:", len(np.unique(gradient_2bit)))
print("4-bit:", len(np.unique(gradient_4bit)))
```

```
8-bit: 256
1-bit: 2
2-bit: 4
4-bit: 16
```

The number of available intensity levels depends on the bit depth:

- 1-bit: 2 intensity levels.
- 2-bit: 4 intensity levels.
- 4-bit: 16 intensity levels.

- 8-bit: 256 intensity levels.

21. Visualizing Different Bit Depths

The original 8-bit gradient and its quantized versions were displayed for comparison.

```
Visualizing Different Bit-Depth Gradients

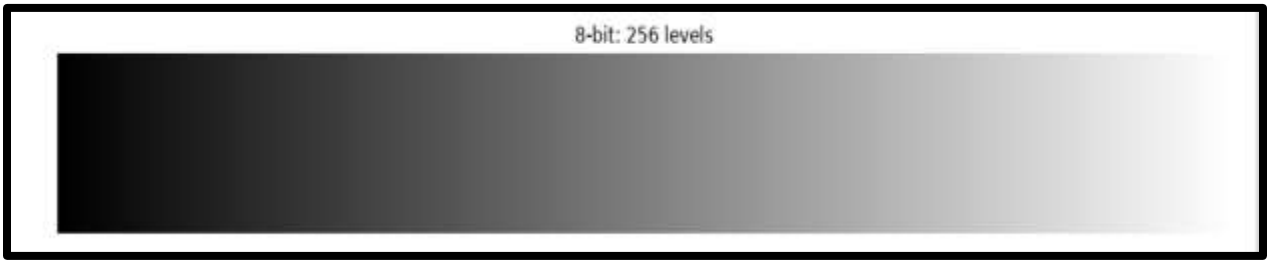
fig, axes = plt.subplots(4, 1, figsize=(12, 7))

gradients = [
    (gradient_image, "8-bit: 256 levels"),
    (np.tile(gradient_1bit, (30, 1)), "1-bit: 2 levels"),
    (np.tile(gradient_2bit, (30, 1)), "2-bit: 4 levels"),
    (np.tile(gradient_4bit, (30, 1)), "4-bit: 16 levels")
]

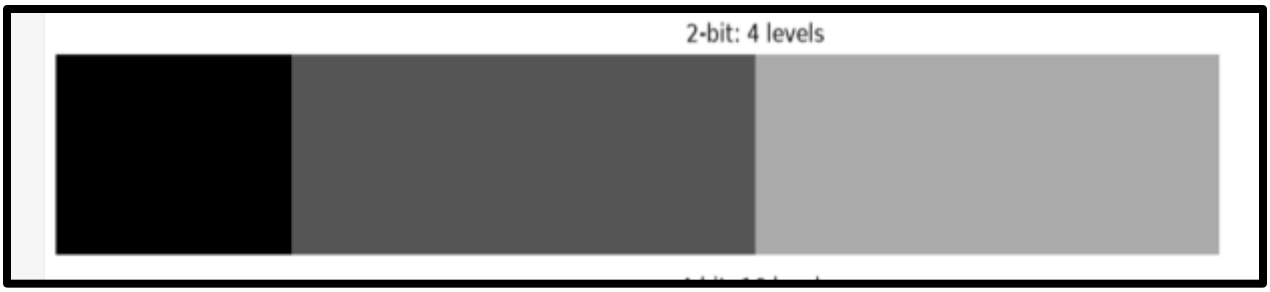
for ax, (data, title) in zip(axes, gradients):
    ax.imshow(
        data,
        cmap="gray",
        vmin=0,
        vmax=255,
        aspect="auto"
    )
    ax.set_title(title)
    ax.axis("off")

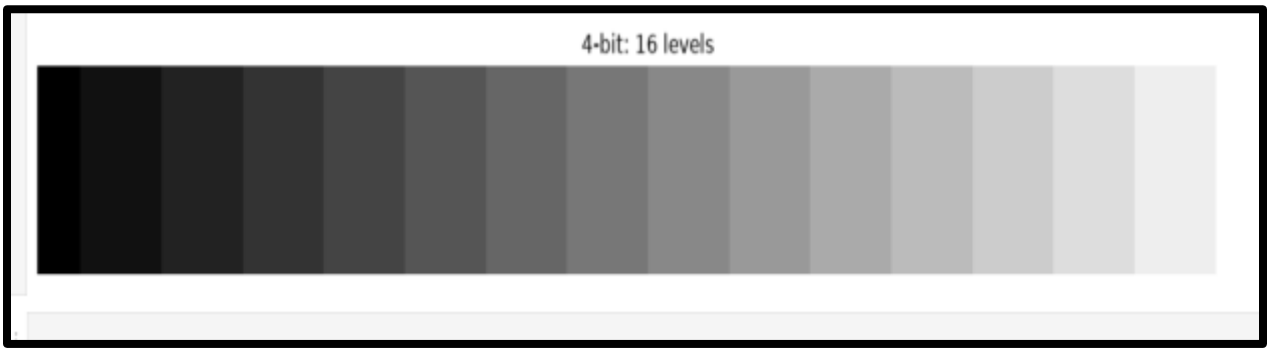
plt.tight_layout()
plt.show()
```

The comparison demonstrates that reducing bit depth decreases the number of available intensity levels and produces more visible transitions between shades.









22. Understanding Image Resolution and Bit Depth

Image Resolution

Image resolution refers to the number of pixels in an image. It is generally represented as width × height. Higher resolution provides more pixels to represent spatial details.

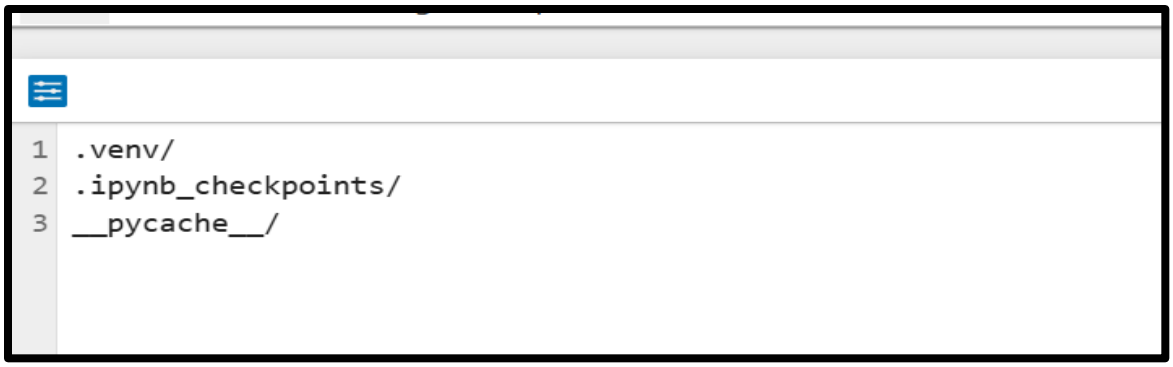
Bit Depth

Bit depth refers to the number of bits used to represent the intensity of each pixel. A higher bit depth provides more possible intensity levels and allows smoother tonal variations.

Difference Between Resolution and Bit Depth

Resolution determines the number of pixels in an image, whereas bit depth determines how many intensity values each pixel can represent. Both properties affect the representation and quality of a digital image.

23. Creating the .gitignore File



```
.venv/
.ipynb_checkpoints/
__pycache__/
```

This helps prevent temporary notebook files and environment folders from being included in a Git repository.

25. Results and Discussion

The laboratory task demonstrated several fundamental digital image processing operations using Python. The input image was loaded and examined to identify its dimensions, data type, pixel values, and color channels.

Color space conversion was performed to obtain RGB and grayscale images. Individual color channels were visualized, and the center pixel's RGB and hexadecimal values were extracted.

Image cropping and resizing operations were successfully performed, and the processed image was saved. A grayscale histogram was generated to study the distribution of pixel intensities. Finally, gradient quantization was used to compare different bit depths.

26. Conclusion

In this laboratory task, the basic concepts of digital images, pixels, color spaces, and image transformations were studied and implemented using Python, OpenCV, NumPy, and Matplotlib in Anaconda Jupyter Notebook.

The practical exercises provided an understanding of how digital images are represented, how their properties can be inspected, and how basic processing operations can be performed. The histogram and bit-depth experiments further demonstrated the importance of intensity distribution and quantization in digital image processing.

Overall, this laboratory task strengthened the understanding of fundamental digital image processing techniques and their practical implementation in Python.